DESIGN AND VERIFICATION OF A 3 STAGE PIPELINED 4 BIT CPU DATAPATH

RTL Project Report

Item	Information
Student name	Vuong Tri Tien
Roll number	HE190717
Project title	3-stage pipelined 4-bit CPU datapath
HDL	Verilog HDL
Development tool	AMD Vivado 2026.1
Target part	xc7a35tftg256-2L
Submission date	2026/10/09

Abstract

This report presents the design, RTL implementation, and verification of a 3-stage pipelined 4-bit CPU datapath developed from an existing 4-bit ALU project. The processor combines a 4-bit program counter, a 16-word instruction memory, IF/ID and ID/EX pipeline registers, a four-register file, the reused ALU, a 16-word data memory, status flags, a write-back multiplexer, and a dedicated CPU control unit. Instructions advance through instruction fetch, instruction decode, and execute stages. After the two-cycle fill interval, the pipeline can complete one instruction per clock.

The instruction set remains compatible with the earlier CPU and includes arithmetic, logic, immediate, register transfer, memory access, and control-flow operations. A self-checking Verilog testbench verifies pipeline fill and overlap, data dependencies, memory transfers, flag behavior, unconditional jumps, taken and non-taken conditional jumps, and two-stage flushing after a taken control transfer. Behavioral simulation in Vivado 2026.1 completes 42 checks with zero errors and reports PASS at approximately 386 ns. Synthesis for xc7a35tftg256-2L also completes with zero errors and zero critical warnings, using 37 LUTs and 60 slice registers.

Contents

- 1 Project Overview
- 2 Design Requirements
- 3 CPU Architecture
- 4 Instruction Set Architecture
- 5 Control and Datapath Operation
- 6 RTL Module Implementation
- 7 Verification Methodology
- 8 Vivado Design Flow
- 9 Results and Discussion
- 10 Limitations and Future Development
- 11 Conclusion
- Appendix A Source File Inventory
- Appendix B Demonstration Program
- Appendix C Vivado Evidence Checklist

1 Project Overview

1 1 Background

The starting point of this project was a modular 4-bit ALU capable of arithmetic, logic, shifting, multiplication, and division functions. An ALU alone transforms input operands but does not fetch instructions, retain a program sequence, store general-purpose data, or make control-flow decisions. The project extends that computational block into a minimal processor datapath by adding state elements, memories, instruction decoding, and control logic.

1 2 Project Objective

The objective is to implement a compact 4-bit CPU that demonstrates instruction fetch, decode, execute, memory, write-back, and branch handling in a visible three-stage pipeline. The design emphasizes understandable module boundaries, valid-bit control, deterministic flushing, and observable signals rather than a large instruction set.

1 3 Scope

- 4-bit data width and 4-bit addresses.
- 16 instructions in program memory and 16 data words in data memory.
- Four general-purpose registers named R0 through R3.
- Twelve implemented CPU opcodes and four reserved opcodes.
- Three pipeline stages: IF, ID, and EX, separated by IF/ID and ID/EX registers.
- One instruction can complete each clock after pipeline fill; taken branches flush two younger instructions.
- Behavioral simulation using a self-checking testbench.
- Synthesized RTL targeting xc7a35tftg256-2L with 37 LUTs, 60 slice registers, zero BRAM tiles, and zero DSPs.

2 Design Requirements

The requirements were derived from the common RTL laboratory workflow and adapted to the CPU datapath project. The implementation must use explicit Verilog modules, provide a repeatable simulation, expose useful debug signals, and produce evidence from the Vivado flow.

Requirement	Specification	Verification method
Data path width	4 bits	Waveform and register checks
Instruction width	12 bits	IF/ID and ID/EX instruction observation
Program storage	16 x 12-bit instruction memory	Program execution sequence
Data storage	16 x 4-bit data memory	STORE and LOAD checks
Register storage	Four 4-bit registers with two read ports and one write port	Register value checks
Arithmetic and logic	ADD, SUB, AND, OR, XOR, and NOT	Directed self-checking programs
Data movement	LDI and MOV	Register write-back checks
Control flow	JMP and JZ with two-younger-instruction flush	Taken and non-taken branch checks
Pipeline behavior	IF, ID, and EX overlap with valid-bit bubble control	Cycle-by-cycle stage checks
Reset behavior	PC, pipeline valid bits, registers, and flags return to known values	Reset before each test
Test outcome	Final PASS when all checks succeed	Tcl Console output

3 CPU Architecture

3 1 Architectural Organization

The processor follows a small Harvard-style organization with separate instruction and data memories. Its datapath is divided into three stages. IF uses the current program counter to read instruction memory. ID stores the fetched instruction and its PC in the IF/ID register. EX receives a second registered copy through the ID/EX register, decodes that instruction, reads the register operands, performs ALU or memory work, commits architectural state, and resolves branches.

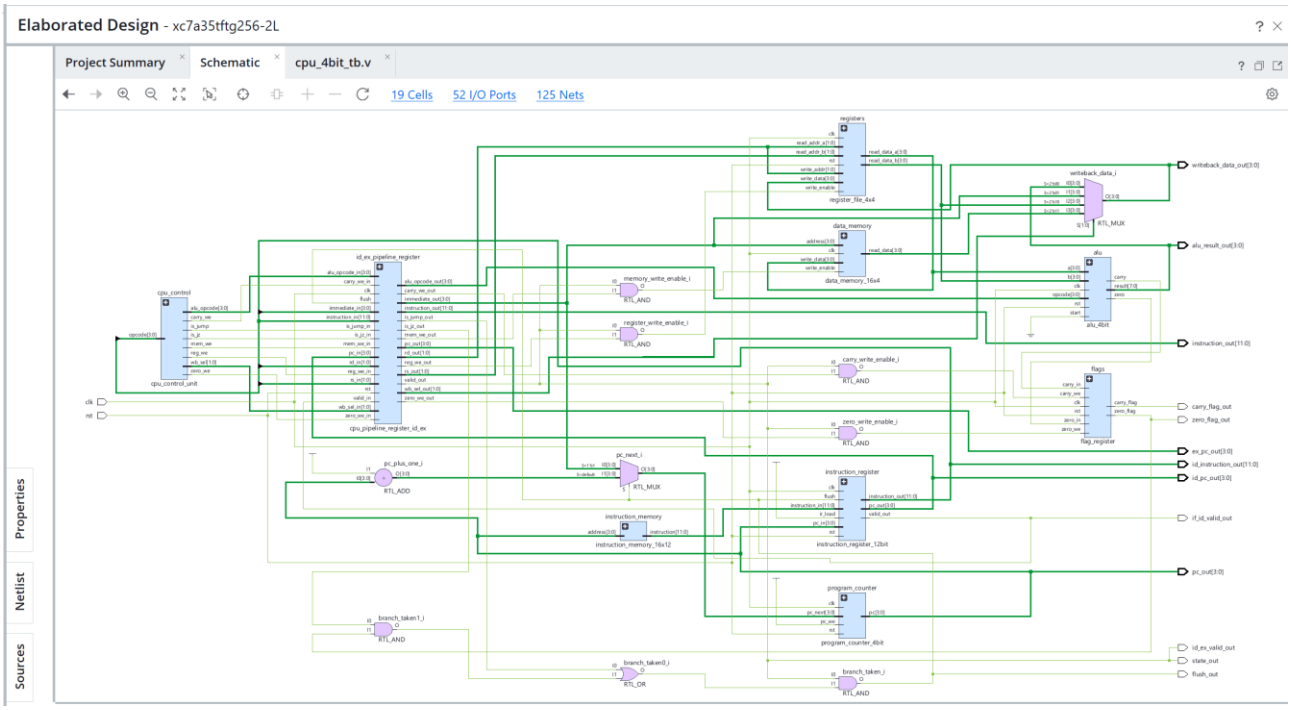


Figure 1 Elaborated RTL architecture of the 3-stage pipelined CPU datapath

3 2 How Information Moves Through the CPU

The architecture can be read as five cooperating paths. The fetch path supplies an instruction and its PC. The two pipeline registers carry instruction identity and validity toward EX. The control path decodes only the valid EX instruction. The data path reads operands, performs an ALU or memory operation, and selects write-back data. The redirect path chooses sequential PC + 1 or the branch target and simultaneously flushes younger instructions after a taken JMP or JZ.

- Fetch path: PC -> instruction memory -> IF/ID instruction and PC fields.
- Pipeline path: IF/ID -> ID/EX, with a valid bit in each register.
- Control path: EX opcode -> CPU control unit -> enables, selections, and branch decision.
- Data path: register operands -> ALU or data memory -> write-back multiplexer -> register file.
- Redirect path: PC + 1 normally; branch target plus IF/ID and ID/EX flush for a taken JMP or JZ.

Register operands are read in EX rather than captured earlier. Consequently, a value written by one instruction is visible to the immediately following instruction without a separate forwarding network or a data-hazard stall in this design. Zero and Carry are stored in a flag register, so JZ can test the architectural Zero value established by an earlier ALU instruction.

3 3 Datapath Components

Block	Width or capacity	Main responsibility
Program counter	4 bits	Addresses IF every clock and loads PC + 1 or an EX-stage branch target.
Instruction memory	16 x 12 bits	Stores the executable program loaded from program.mem.
IF/ID register	12-bit instruction, 4-bit PC, valid	Captures each fetched instruction and can be cleared by reset or flush.
ID/EX register	12-bit instruction, 4-bit PC, valid	Transfers the decoded-stage entry to EX and inserts a bubble on flush.
Register file	4 x 4 bits	Provides two asynchronous read ports and one synchronous write port.
ALU	4-bit operands, 8-bit result bus	Executes arithmetic and logic operations reused from the ALU project.
Data memory	16 x 4 bits	Supports asynchronous reads and synchronous writes.
Flag register	Zero and Carry	Stores status required by later instructions, including JZ.
Write-back multiplexer	4 bits	Selects ALU, immediate, source-register, or memory data.
CPU control unit	EX opcode decoder	Generates write enables, ALU opcode, write-back selection, and branch controls.

3 4 Instruction Format

Every instruction is twelve bits wide. The same physical fields are interpreted according to the opcode. Register-format operations use both register fields, while immediate, memory, and branch instructions use the low four bits as an immediate value or address.

Bits	Field	Description
11 to 8	opcode	Selects the CPU instruction.
7 to 6	Rd	Destination register or store-data register.
5 to 4	Rs	Second source register for register-format instructions.
3 to 0	immediate or address	Immediate data, data-memory address, or branch target.

4 Instruction Set Architecture

The CPU defines opcodes 0 through B. Opcodes C through F are reserved and behave as NOP because the decoder uses safe default control signals. This also prevents CPU LOAD and STORE opcodes from accidentally launching the sequential multiply or divide functions of the reused ALU.

Opcode	Assembly form	Operation	Flag effect
0	ADD Rd, Rs	Rd <- Rd + Rs	Z and C
1	SUB Rd, Rs	Rd <- Rd - Rs	Z and C
2	AND Rd, Rs	Rd <- Rd AND Rs	Z
3	OR Rd, Rs	Rd <- Rd OR Rs	Z
4	XOR Rd, Rs	Rd <- Rd XOR Rs	Z
5	NOT Rd	Rd <- NOT Rd	Z
6	LDI Rd, imm	Rd <- immediate	Unchanged
7	MOV Rd, Rs	Rd <- Rs	Unchanged
8	LOAD Rd, addr	Rd <- data_memory[addr]	Unchanged
9	STORE Rd, addr	data_memory[addr] <- Rd	Unchanged
A	JMP addr	PC <- address	Unchanged
B	JZ addr	If Z = 1 then PC <- address else PC <- PC + 1	Read Z
C to F	Reserved	No architectural state change other than PC advance	Unchanged

5 Control and Datapath Operation

5 1 Control Signal Table

Instruction	wb_sel	reg_we	mem_we	zero_we	carry_we	is_jump	is_jz
ADD	ALU	1	0	1	1	0	0
SUB	ALU	1	0	1	1	0	0
AND	ALU	1	0	1	0	0	0
OR	ALU	1	0	1	0	0	0
XOR	ALU	1	0	1	0	0	0
NOT	ALU	1	0	1	0	0	0
LDI	IMM	1	0	0	0	0	0
MOV	RS	1	0	0	0	0	0
LOAD	MEM	1	0	0	0	0	0
STORE	ALU	0	1	0	0	0	0
JMP	ALU	0	0	0	0	1	0
JZ	ALU	0	0	0	0	0	1
Reserved	ALU	0	0	0	0	0	0

5 2 Pipeline Timing and Valid Bits

Condition	IF action	IF/ID	ID/EX	Architectural effect
Reset	PC = 0	IF/ID valid = 0	ID/EX valid = 0	Registers and flags cleared
First clock	Fetch PC 0	Instruction 0 valid	Bubble	No architectural commit
Second clock	Fetch PC 1	Instruction 1 valid	Instruction 0 valid	Pipeline still filling
Third and later	Fetch PC n	Instruction n valid	Instruction n-1 valid	Valid EX instruction may commit
Taken branch	Load target	Flushed to invalid	Flushed to invalid	Branch instruction commits; two younger instructions are discarded

The valid bits distinguish real instructions from bubbles. Register, data-memory, flag, and branch side effects are enabled only when ID/EX valid is high. state_out is retained for compatibility with the earlier testbench interface, but in the pipelined design it reports EX validity rather than a FETCH/EXEC controller state.

5 3 Instruction Walkthrough: ADD

Machine word 060 represents ADD R1, R2. At address 2, IF reads 060 and the first pipeline register captures both the word and PC = 2. On the next edge, the entry transfers to ID/EX while the following instruction enters IF/ID. When ADD reaches EX, Rd selects R1 = 5 and Rs selects R2 = 3. The control unit selects ADD and enables register and flag writes. The ALU produces 8, the write-back multiplexer selects the low four ALU bits, R1 becomes 8, Z becomes 0, and C becomes 0.

Stage	Active path	Effect
IF	PC 2 -> instruction memory -> IF/ID	IF/ID receives 060 with PC = 2 and valid = 1.
ID	IF/ID -> ID/EX	Instruction, PC, and valid bit advance; younger work proceeds in parallel.
EX	R1 and R2 -> ALU -> write-back	5 + 3 = 8; R1 receives 8; Z = 0; C = 0.
Overlap	PC 4 in IF while PC 3 is in ID	Pipeline sustains one instruction per clock after fill.

5 4 Instruction Walkthrough: STORE and LOAD

The pair 940 and 800 demonstrates the memory path. When STORE R1,[0] is valid in EX, it reads R1 and asserts the data-memory write enable, causing value 8 to be stored at address 0. LOAD R0,[0] then reads the same memory location asynchronously, selects MEM at the write-back multiplexer, and writes 8 into R0 when it reaches EX. STORE does not modify a register or flags, while LOAD modifies only its destination register.

5 5 Instruction Walkthrough: SUB and JZ

The sequence 1A0 followed by B09 demonstrates status-dependent control flow. SUB R2,R2 computes 3 - 3 = 0, writes zero into R2, sets Z to 1, and sets C to 1 according to the ALU's no-borrow convention. When JZ 9 later reaches EX, the control unit combines is_jz with the stored Zero flag. A true branch redirects PC to 9 and clears both IF/ID and ID/EX, so the two younger sequential instructions become bubbles and cannot change architectural state.

5 6 Next Program Counter Logic

```
branch_taken = is_jump OR (is_jz AND zero_flag)
pc_next = branch_taken ? immediate : pc + 1
pipeline_flush = ex_valid AND branch_taken
```

JMP always selects the low four EX-instruction bits as the target. JZ selects the same target only when the stored Zero flag is set. Because the decision is made in EX, two younger sequential instructions may already occupy IF/ID and ID/EX. pipeline_flush invalidates both entries on the redirect edge, producing a deterministic two-cycle taken-branch penalty. A non-taken JZ does not flush and sequential overlap continues.

6 RTL Module Implementation

6 1 New CPU Modules

Source file	Purpose
program_counter_4bit.v	Synchronous 4-bit PC with reset and write enable.
instruction_memory_16x12.v	Program ROM initialized from program.mem; unused locations contain C00 NOP.
instruction_register_12bit.v	IF/ID register carrying instruction, PC, and valid, with reset and flush support.
register_file_4x4.v	Four explicit 4-bit registers with two asynchronous reads and one synchronous write.
data_memory_16x4.v	Sixteen 4-bit words with asynchronous read and synchronous write.
flag_register.v	Stores Zero and Carry with independent write enables.
cpu_control_unit.v	Combinational CPU opcode decoder and safe defaults for reserved instructions.
cpu_4bit_top.v	Integrates the datapath, control, IF/ID and ID/EX sequencing, branch flush, and debug outputs. It also contains the internal cpu_pipeline_register_id_ex module.

6 2 Reused ALU Modules

Source file	Purpose in the integrated design
full_adder.v	Single-bit full-adder primitive.
adder_subtractor_4bit.v	Shared ripple-carry addition and subtraction path.
logic_unit_4bit.v	AND, OR, XOR, and NOT result generation.
shifter_4bit.v	Shift-left and shift-right functions retained in the ALU.
muldiv_unit.v	Sequential multiply and divide unit retained but not launched by the base CPU.
flag_unit.v	Produces ALU status outputs.
control_unit.v	Decodes the internal ALU opcode.
alu_4bit.v	Top-level reused ALU instance.

6 3 Important Implementation Decisions

- The ALU start input is tied low because the first CPU version does not expose the sequential multiply and divide operations.
- Only the low four bits of the ALU result are written to the register file.
- Zero is updated by arithmetic and logic instructions; Carry is updated only by ADD and SUB.
- The data memory has no explicit reset input. It is initialized to zero in simulation and written only by a valid EX-stage STORE.
- Architectural side effects are gated by ID/EX valid, so reset bubbles and flushed instructions cannot write registers, memory, or flags.
- Register operands are read in EX. This keeps back-to-back dependencies correct without adding forwarding or stall logic to the present pipeline.
- Debug outputs expose IF PC, ID PC, EX PC, IF/ID instruction, ID/EX instruction, both valid bits, flush, ALU result, write-back data, Zero, and Carry.

7 Verification Methodology

7 1 Self Checking Testbench

The cpu_4bit_tb testbench generates a 10 ns clock, applies active-high reset before each test, loads directed instruction sequences into instruction memory, clears data memory where required, and compares pipeline or architectural signals with expected values. It first checks reset, fill, IF/ID/EX overlap, and valid-bit behavior. Directed programs then verify data dependencies, memory, flags, branch redirects, and flush protection. Each comparison increments a test counter and reports PASS or ERROR.

```
Student name: Vuong Tri Tien
Roll number: HE190717
PASS: ALL 42 PIPELINED CPU CHECKS PASSED
```

The exact console screenshot is reserved for Appendix C so that the main verification section can concentrate on what was checked and how the observed signals prove correct operation.

7 2 Verification Scenarios

ID	Check	Expected	Observed	Status
1	PC after reset	0	0	PASS
2	IF/ID after reset	Invalid	Invalid	PASS
3	ID/EX after reset	Invalid	Invalid	PASS
4	First clock ID PC	0	0	PASS
5	First fetch IF/ID valid	1	1	PASS
6	First fetch EX bubble	Invalid	Invalid	PASS
7	Overlap IF PC	2	2	PASS
8	Overlap ID PC	1	1	PASS
9	Overlap EX PC	0	0	PASS
10	EX valid after fill	1	1	PASS
11	ADD R1,R2	R1 = 8	8	PASS
12	ADD source preservation	R2 = 3	3	PASS
13	ADD Zero flag	Z = 0	0	PASS
14	ADD Carry flag	C = 0	0	PASS
15	XOR R1,R3	R1 = 2	2	PASS
16	LDI R3,A	R3 = A	A	PASS
17	XOR Zero flag	Z = 0	0	PASS
18	JMP redirect	PC = 5	5	PASS
19	JMP flush IF/ID	Invalid	Invalid	PASS
20	JMP flush ID/EX	Invalid	Invalid	PASS
21	STORE R1,[0]	MEM[0] = 8	8	PASS
22	LOAD R0,[0]	R0 = 8	8	PASS
23	SUB R2,R2	R2 = 0	0	PASS
24	SUB Zero flag	Z = 1	1	PASS
25	SUB no-borrow Carry	C = 1	1	PASS
26	Taken JZ redirect	PC = 9	9	PASS

ID	Check	Expected	Observed	Status
27	JZ flush IF/ID	Invalid	Invalid	PASS
28	JZ flush ID/EX	Invalid	Invalid	PASS
29	MOV R3,R0	R3 = 8	8	PASS
30	Taken-branch skipped writes	R3 remains 8	8	PASS
31	AND C,A	R0 = 8	8	PASS
32	OR 8,A	R0 = A	A	PASS
33	NOT A	R0 = 5	5	PASS
34	NOT Zero flag	Z = 0	0	PASS
35	Logic preserves Carry	C = 0	0	PASS
36	Non-taken JZ flush	0	0	PASS
37	Instruction after JZ reaches EX	EX PC = 6	6	PASS
38	XOR 5,A	R0 = F	F	PASS
39	JMP skips address 8	PC = 9	9	PASS
40	JMP creates EX bubble	Invalid	Invalid	PASS
41	MOV R2,R0	R2 = F	F	PASS
42	Flushed address 8 cannot write	R2 remains F	F	PASS

7 3 Waveform Evidence

The updated waveform should show IF, ID, and EX operating concurrently after reset. During steady execution, pc_out advances every clock while id_pc_out and ex_pc_out trail it by one and two stages. if_id_valid_out and id_ex_valid_out identify real work; flush_out asserts for one redirect edge and clears both younger stages. state_out now mirrors EX validity for compatibility.

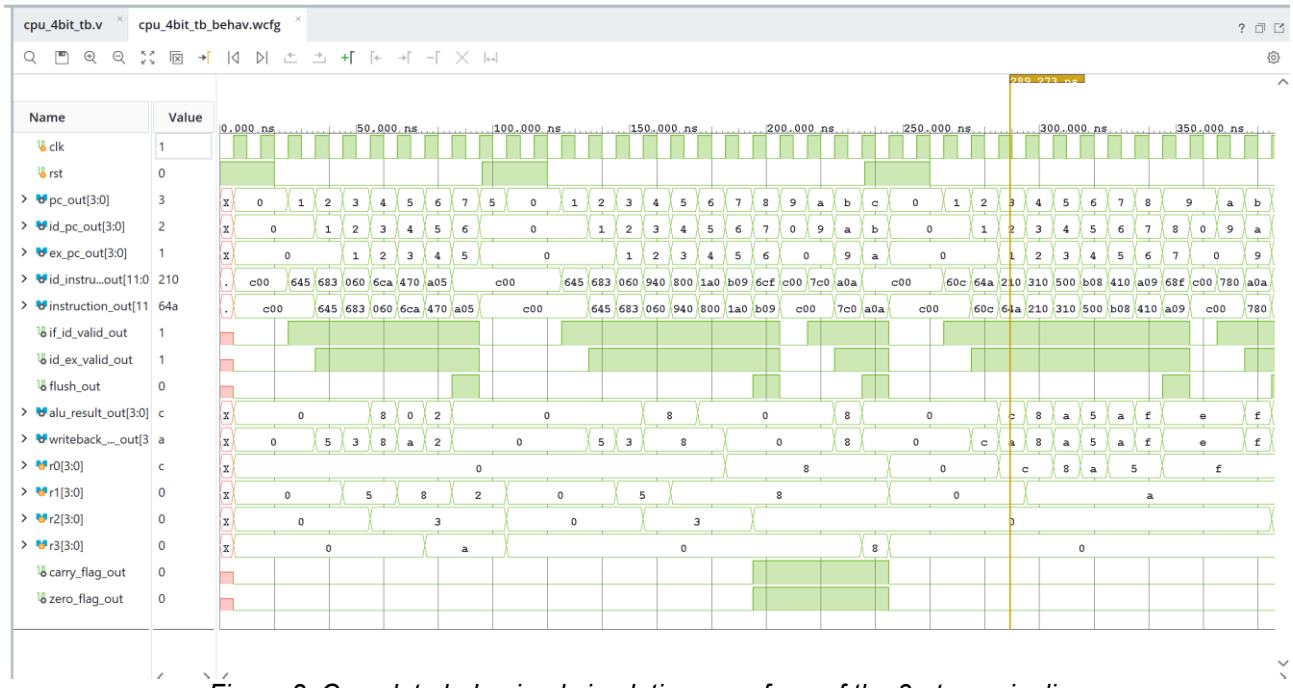


Figure 2 Complete behavioral simulation waveform of the 3-stage pipeline

The verified memory-and-branch sequence stores 8 with 940, loads 8 with 800, and uses 1A0 to produce Z = 1 and no-borrow C = 1. When B09 reaches EX, PC redirects to 9 and the two valid bits

clear, proving that younger instructions were flushed. Across the full run, tests reaches hexadecimal 2A, equal to 42 decimal checks, errors remains 00000000, and simulation finishes at approximately 386 ns.

8 Vivado Design Flow

The project follows the standard RTL flow used in Vivado. Behavioral simulation and RTL synthesis have been completed for the pipelined design. program.mem is included in sources_1 as a Memory Initialization File, and the synthesis log confirms successful readmem loading. Implementation, timing analysis with a clock constraint, and DRC have not yet been run; their results must not be inferred from synthesis alone.

Step	Vivado action	Purpose	Required evidence
1	Create or open project	Establish the target device, source sets, and project hierarchy.	Project name and Sources window
2	Add RTL and simulation sources	Compile the CPU, reused ALU, testbench, and program.mem.	Design Sources and Simulation Sources
3	Run behavioral simulation	Verify functional behavior before synthesis.	Waveform and Tcl Console PASS
4	Open elaborated design	Inspect inferred hierarchy, buses, muxes, registers, and connectivity.	RTL schematic
5	Run synthesis	Map RTL to FPGA primitives and report warnings and utilization.	Synthesis status and utilization
6	Review timing and DRC	Check design-rule and timing feasibility for implementation.	Timing summary and DRC

Project hierarchy, Tcl Console, synthesis, utilization, timing, and DRC screenshots are placed in Appendix C. Keeping these supporting images after the technical discussion makes the main report easier to read while preserving a clear evidence trail.

8 1 Synthesis Result Table

Metric	Result	Source in Vivado
Synthesis status	Complete: 0 errors, 0 critical warnings	Design Runs and synthesis log
Slice LUTs	37 (0.18%)	Report Utilization
Slice registers	60 (0.14%)	Report Utilization
Block RAM tiles	0	Report Utilization
DSPs	0	Report Utilization
Bonded IOBs	52 (30.59%)	Report Utilization
Worst negative slack	Not available: no timing constraints or implementation run	Report Timing Summary pending
DRC status	Not available: implementation not run	Report DRC pending

9 Results and Discussion

9 1 Functional Result

Behavioral simulation completed successfully. All 42 directed checks passed, the error counter remained zero, and the final testbench message confirmed the overall PASS result. The verified scenarios cover pipeline reset and fill, three-stage overlap, ALU execution, immediate loading, register movement, data-memory write and read, flag persistence, unconditional jumps, conditional branches in both directions, and protection against writes by flushed instructions.

9 2 Interpretation

The test results demonstrate that the data path, pipeline control, and architectural side effects are coordinated correctly. Write enables are qualified by EX validity, so bubbles and flushed instructions cannot modify registers, data memory, or flags. The stored Zero flag is available to JZ, and a taken control transfer redirects the PC while invalidating both younger stages.

Compared with the earlier two-state implementation, the pipeline raises steady-state throughput from one instruction every two clocks to one instruction per clock after fill. The trade-off is explicit control-hazard handling: a taken JMP or JZ has a two-cycle penalty because its decision is resolved in EX. Reading register operands in EX keeps the current back-to-back data dependencies correct without forwarding hardware.

9 3 Reset and Memory Behavior

Synchronous reset initializes the PC, both pipeline registers and their valid bits, the register file, and stored flags. Data memory does not have a reset port; it is initialized by an initial block and explicitly cleared by the testbench when required. This behavior is acceptable for the present FPGA-oriented demonstration but must be considered if the design is migrated to a different target or ASIC flow.

10 Limitations and Future Development

The current CPU intentionally limits functionality to keep the datapath and control flow easy to inspect. The following items define the boundary of the submitted version and practical directions for later development.

- Expose shift-left and shift-right instructions in the CPU instruction set.
- Integrate sequential multiplication and division with start, busy, and done handshaking.
- Store or expose additional ALU flags such as Negative and Overflow.
- Add a HALT instruction instead of using a self-looping JMP at the end of a program.
- Add an assembler or a script that converts assembly mnemonics into 12-bit program words.
- Create an FPGA wrapper with clock division, reset debouncing, switches, LEDs, and seven-segment output.
- Add an XDC clock constraint, run implementation, and evaluate WNS, routing, and DRC before FPGA deployment.
- If operand reads are moved into ID in a future version, add forwarding and stall logic for read-after-write hazards.
- Remove unused simulation sources before archiving the final project and verify the archive on a clean directory.

11 Conclusion

A functional 3-stage pipelined 4-bit CPU datapath was created by extending the existing ALU with IF/ID and ID/EX sequencing, valid-bit bubble control, register storage, program and data memories, status storage, write-back selection, and EX-stage branch resolution. The processor retains the twelve-instruction ISA while allowing one instruction to complete per clock after pipeline fill.

The self-checking Vivado simulation provides direct evidence that arithmetic, logic, data transfer, memory access, dependencies, and branch flushing operate correctly. All 42 checks pass at approximately 386 ns after the testbench prints the required student name and roll number. Synthesis completes with 37 LUTs, 60 slice registers, zero BRAM tiles, and zero DSPs. Final submission work is limited to updated Vivado screenshots and, if required, constrained implementation evidence for timing and DRC.

References

1. Digital System Design Lab assignment brief, Verilog HDL and Vivado, common project requirements and marking scheme.
2. AMD Vivado Design Suite 2026.1, behavioral simulation, RTL elaboration, synthesis, and reporting workflow.
3. 4bit RISC CPU Pipeline Report, three-stage IF/ID/EX organization and branch-flush reference.
4. Verilog HDL source files developed and integrated for the 3-stage pipelined 4-bit CPU Datapath project.

Appendix A Source File Inventory

Category	File or files
CPU top	cpu_4bit_top.v
CPU control	cpu_control_unit.v
Program counter	program_counter_4bit.v
Instruction memory	instruction_memory_16x12.v
IF/ID pipeline register	instruction_register_12bit.v
ID/EX pipeline register	cpu_pipeline_register_id_ex inside cpu_4bit_top.v
Register file	register_file_4x4.v
Data memory	data_memory_16x4.v
Stored flags	flag_register.v
Reused ALU top	alu_4bit.v
Reused ALU modules	control_unit.v, logic_unit_4bit.v, shifter_4bit.v, adder_subtractor_4bit.v, full_adder.v, muldiv_unit.v, flag_unit.v
Simulation	cpu_4bit_tb.v
Program image	program.mem

Submission Packaging Checklist

- Copy all active design and simulation sources into the CPU project directory.
- Remove the unused alu_4bit_tb.v reference from Simulation Sources if it is not part of the submission.
- Include program.mem and confirm its file type is Memory Initialization File.
- Set cpu_4bit_top as the design top and cpu_4bit_tb as the simulation top.
- Run behavioral simulation and capture the final PASS output.
- Run behavioral simulation and synthesis, then save the updated pipeline waveform, schematic, console, and utilization screenshots.
- If timing and DRC evidence is required, add constraints and run implementation before capturing those reports.
- Use Vivado Project Archive or verify the ZIP in a clean directory before submission.

Appendix B Demonstration Program

Address	Machine code	Assembly	Effect
0	645	LDI R1, 5	R1 <- 5
1	683	LDI R2, 3	R2 <- 3
2	060	ADD R1, R2	R1 <- 8
3	6CA	LDI R3, A	R3 <- A
4	470	XOR R1, R3	R1 <- 2
5	A05	JMP 5	Repeat at address 5

Unused instruction-memory locations are initialized to C00, which is a reserved no-operation encoding in this CPU. The self-checking testbench replaces instruction-memory contents with dedicated programs for memory, branch, and logic verification.

Appendix C Vivado Evidence Checklist

This appendix is reserved for supplementary Vivado evidence. The earlier schematic and waveform belonged to the pre-pipeline CPU and are intentionally not reused as current evidence. Insert new screenshots from the 3-stage project at the marked positions below.

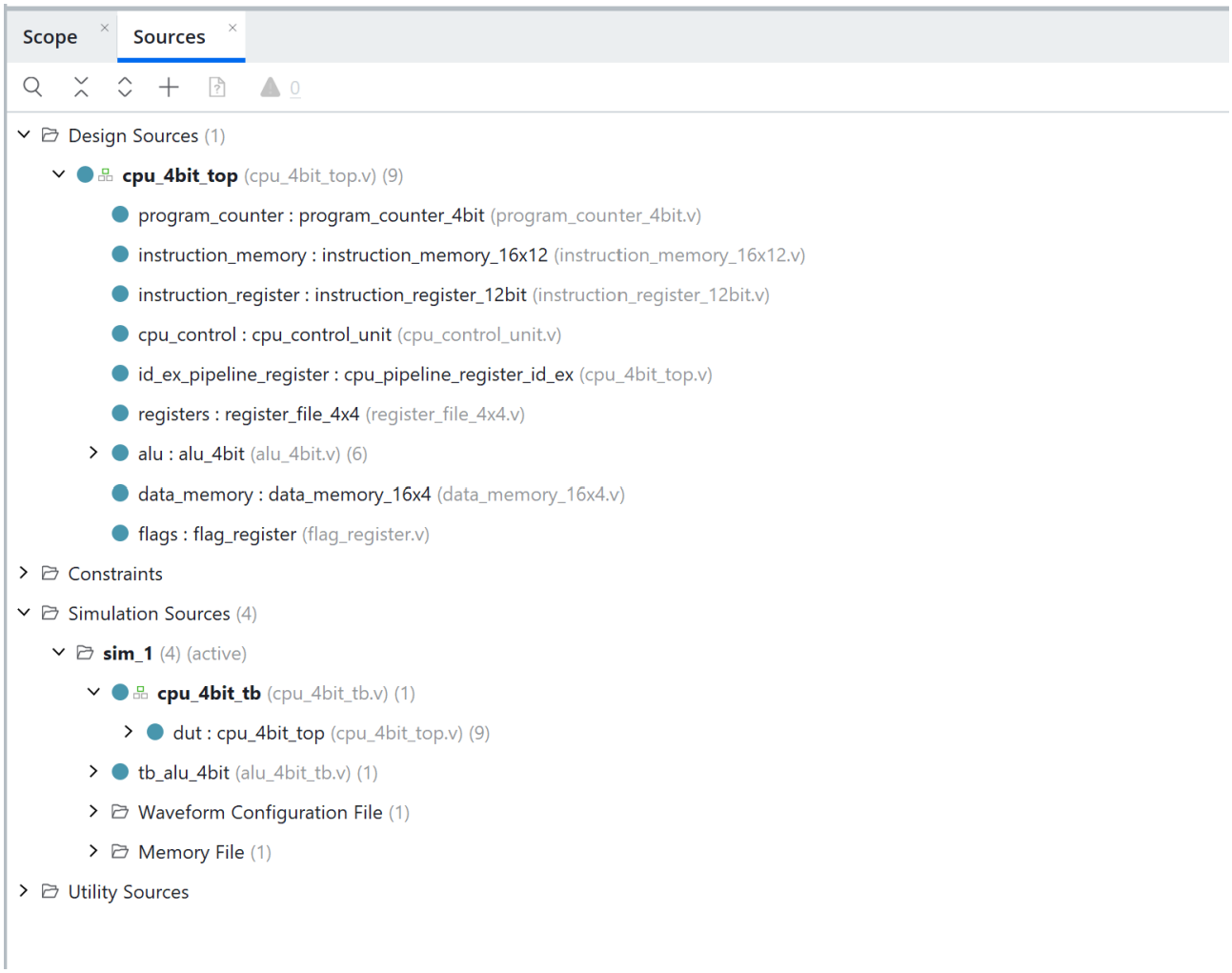


Figure C1 Vivado project hierarchy and selected top modules

```
Student name: Vuong Tri Tien
Roll number: HE190717
Architecture: 3-stage IF/ID/EX pipeline

===== TEST 1: pipeline fill and demo program =====
IF pc=0 | ID valid=0 pc=0 instr=c00 | EX valid=0 pc=0 instr=c00 | WB=0 Z=0 C=0
PASS                                PC is zero after reset actual=0
PASS                              IF/ID is empty after reset actual=0
PASS                              ID/EX is empty after reset actual=0
PASS                    First clock fetches address 0 into ID actual=0
PASS                         IF/ID is valid after first fetch actual=1
PASS                   EX is still a bubble after first fetch actual=0
IF pc=1 | ID valid=1 pc=0 instr=645 | EX valid=0 pc=0 instr=c00 | WB=0 Z=0 C=0
PASS               Pipeline overlap: IF is fetching address 2 actual=2
PASS                     Pipeline overlap: ID holds address 1 actual=1
PASS                     Pipeline overlap: EX holds address 0 actual=0
PASS           EX becomes valid after two pipeline transfers actual=1
IF pc=2 | ID valid=1 pc=1 instr=683 | EX valid=1 pc=0 instr=645 | WB=5 Z=0 C=0
IF pc=3 | ID valid=1 pc=2 instr=060 | EX valid=1 pc=1 instr=683 | WB=3 Z=0 C=0
IF pc=4 | ID valid=1 pc=3 instr=6ca | EX valid=1 pc=2 instr=060 | WB=8 Z=0 C=0
PASS                                  ADD R1,R2 gives R1=8 actual=8
PASS                                          R2 remains 3 actual=3
PASS                                ADD Zero flag is clear actual=0
PASS                               ADD Carry flag is clear actual=0
IF pc=5 | ID valid=1 pc=4 instr=470 | EX valid=1 pc=3 instr=6ca | WB=a Z=0 C=0
IF pc=6 | ID valid=1 pc=5 instr=a05 | EX valid=1 pc=4 instr=470 | WB=2 Z=0 C=0
PASS                                  XOR R1,R3 gives R1=2 actual=2
PASS                                       LDI loaded R3=A actual=a
PASS                                XOR Zero flag is clear actual=0
IF pc=7 | ID valid=1 pc=6 instr=c00 | EX valid=1 pc=5 instr=a05 | WB=0 Z=0 C=0
PASS                           JMP 5 redirects fetch PC to 5 actual=5
PASS                                Taken JMP flushes IF/ID actual=0
PASS                                Taken JMP flushes ID/EX actual=0
```

```
===== TEST 2: memory and taken branch =====
IF pc=0 | ID valid=0 pc=0 instr=c00 | EX valid=0 pc=0 instr=c00 | WB=0 Z=0 C=0
IF pc=1 | ID valid=1 pc=0 instr=645 | EX valid=0 pc=0 instr=c00 | WB=0 Z=0 C=0
IF pc=2 | ID valid=1 pc=1 instr=683 | EX valid=1 pc=0 instr=645 | WB=5 Z=0 C=0
IF pc=3 | ID valid=1 pc=2 instr=060 | EX valid=1 pc=1 instr=683 | WB=3 Z=0 C=0
IF pc=4 | ID valid=1 pc=3 instr=940 | EX valid=1 pc=2 instr=060 | WB=8 Z=0 C=0
IF pc=5 | ID valid=1 pc=4 instr=800 | EX valid=1 pc=3 instr=940 | WB=8 Z=0 C=0
PASS                              STORE wrote 8 to data_memory[0] actual=8
IF pc=6 | ID valid=1 pc=5 instr=1a0 | EX valid=1 pc=4 instr=800 | WB=8 Z=0 C=0
PASS                              LOAD copied data_memory[0] to R0 actual=8
IF pc=7 | ID valid=1 pc=6 instr=b09 | EX valid=1 pc=5 instr=1a0 | WB=0 Z=0 C=0
PASS                                      SUB R2,R2 gives zero actual=0
PASS                                         SUB set Zero flag actual=1
PASS                                   SUB set Carry as no-borrow actual=1
IF pc=8 | ID valid=1 pc=7 instr=6cf | EX valid=1 pc=6 instr=b09 | WB=0 Z=1 C=1
PASS                              Taken JZ redirects fetch PC to 9 actual=9
PASS                                    Taken JZ flushes IF/ID actual=0
PASS                                    Taken JZ flushes ID/EX actual=0
IF pc=9 | ID valid=0 pc=0 instr=c00 | EX valid=0 pc=0 instr=c00 | WB=0 Z=1 C=1
IF pc=a | ID valid=1 pc=9 instr=7c0 | EX valid=0 pc=0 instr=c00 | WB=0 Z=1 C=1
IF pc=b | ID valid=1 pc=a instr=a0a | EX valid=1 pc=9 instr=7c0 | WB=8 Z=1 C=1
PASS                                      MOV R3,R0 gives R3=8 actual=8
PASS                    Instructions at 7 and 8 did not change R3 actual=8
```

```
===== TEST 3: logic and not-taken branch =====
IF pc=0 | ID valid=0 pc=0 instr=c00 | EX valid=0 pc=0 instr=c00 | WB=0 Z=0 C=0
IF pc=1 | ID valid=1 pc=0 instr=60c | EX valid=0 pc=0 instr=c00 | WB=0 Z=0 C=0
IF pc=2 | ID valid=1 pc=1 instr=64a | EX valid=1 pc=0 instr=60c | WB=c Z=0 C=0
IF pc=3 | ID valid=1 pc=2 instr=210 | EX valid=1 pc=1 instr=64a | WB=a Z=0 C=0
IF pc=4 | ID valid=1 pc=3 instr=310 | EX valid=1 pc=2 instr=210 | WB=8 Z=0 C=0
PASS                                         AND C,A gives R0=8 actual=8
IF pc=5 | ID valid=1 pc=4 instr=500 | EX valid=1 pc=3 instr=310 | WB=a Z=0 C=0
PASS                                          OR 8,A gives R0=A actual=a
IF pc=6 | ID valid=1 pc=5 instr=b08 | EX valid=1 pc=4 instr=500 | WB=5 Z=0 C=0
PASS                                           NOT A gives R0=5 actual=5
PASS                                  NOT result keeps Zero clear actual=0
PASS                   Logic operation preserved previous Carry actual=0
IF pc=7 | ID valid=1 pc=6 instr=410 | EX valid=1 pc=5 instr=b08 | WB=a Z=0 C=0
PASS                      Not-taken JZ does not request a flush actual=0
PASS                  Instruction after not-taken JZ reaches EX actual=6
IF pc=8 | ID valid=1 pc=7 instr=a09 | EX valid=1 pc=6 instr=410 | WB=f Z=0 C=0
PASS                                         XOR 5,A gives R0=F actual=f
IF pc=9 | ID valid=1 pc=8 instr=68f | EX valid=1 pc=7 instr=a09 | WB=e Z=0 C=0
PASS                                JMP 9 redirects fetch PC to 9 actual=9
PASS                                   JMP 9 creates an EX bubble actual=0
IF pc=9 | ID valid=0 pc=0 instr=c00 | EX valid=0 pc=0 instr=c00 | WB=e Z=0 C=0
IF pc=a | ID valid=1 pc=9 instr=780 | EX valid=0 pc=0 instr=c00 | WB=e Z=0 C=0
IF pc=b | ID valid=1 pc=a instr=a0a | EX valid=1 pc=9 instr=780 | WB=f Z=0 C=0
PASS                                       MOV R2,R0 gives R2=F actual=f
PASS                          Flushed address 8 did not write R2 actual=f
PASS: ALL 42 PIPELINED CPU CHECKS PASSED
```

Figure C2 Self-checking testbench result in the Vivado Tcl Console

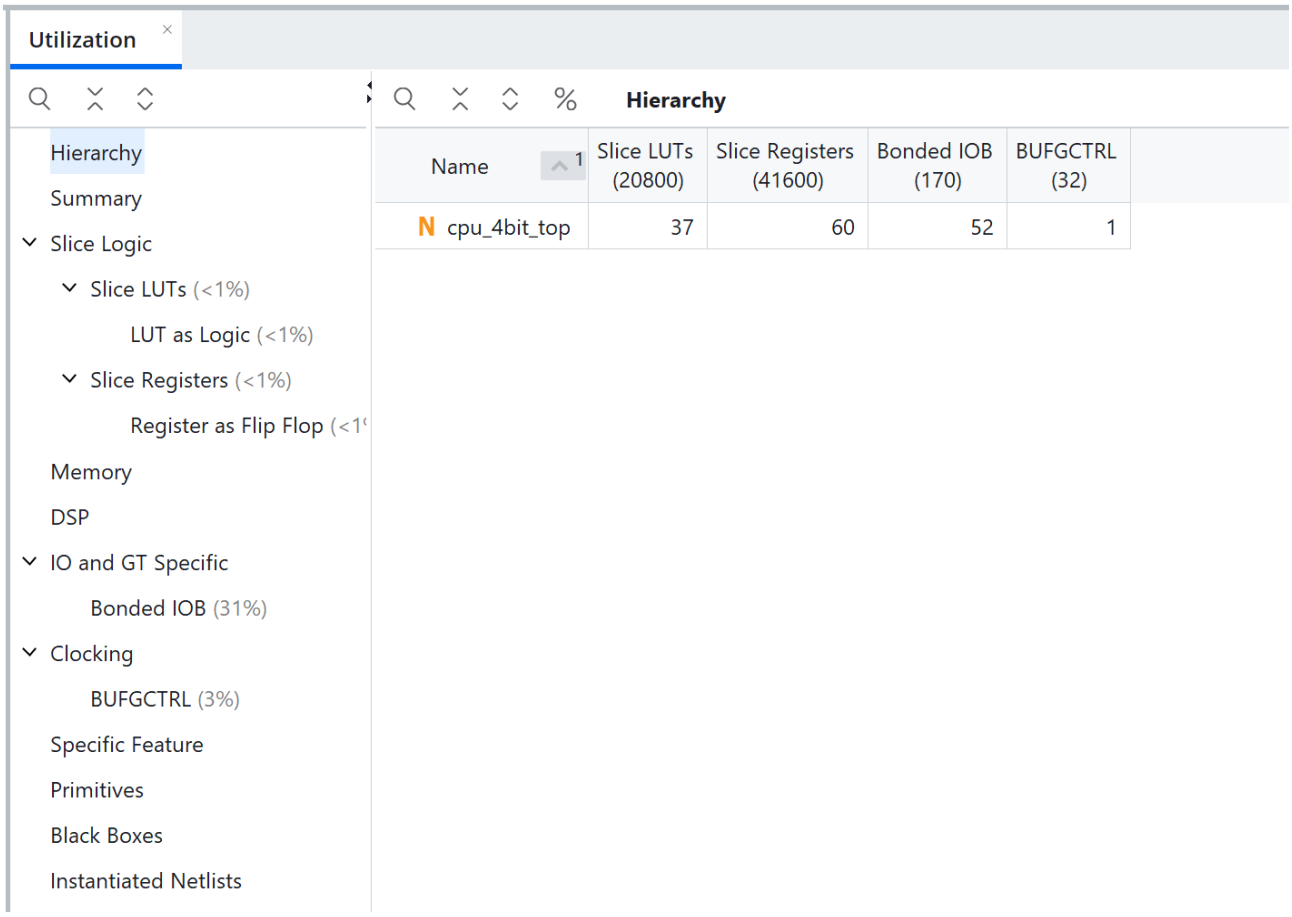


Figure C3 Post-synthesis utilization report

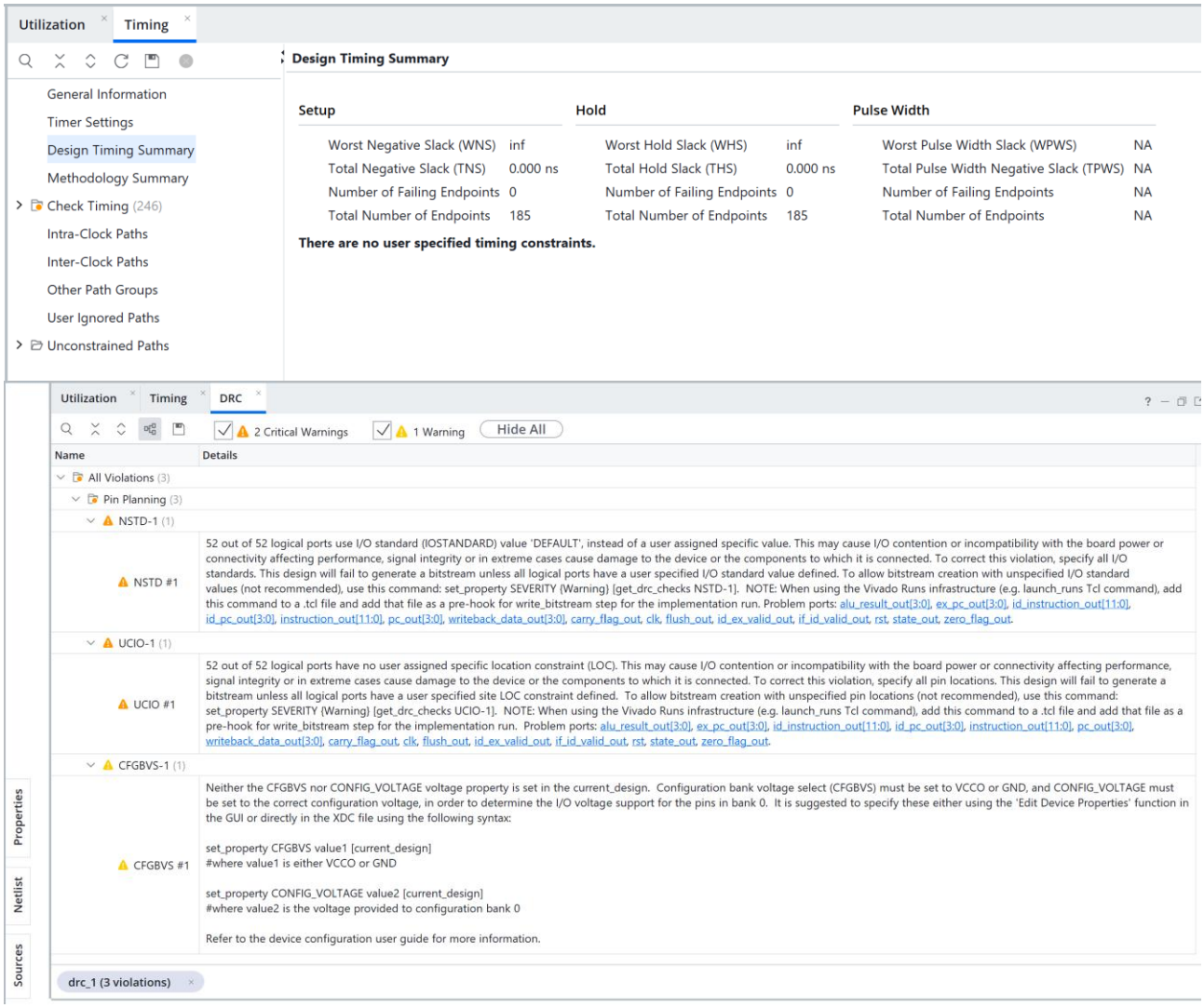


Figure C4 Timing summary and design-rule check evidence